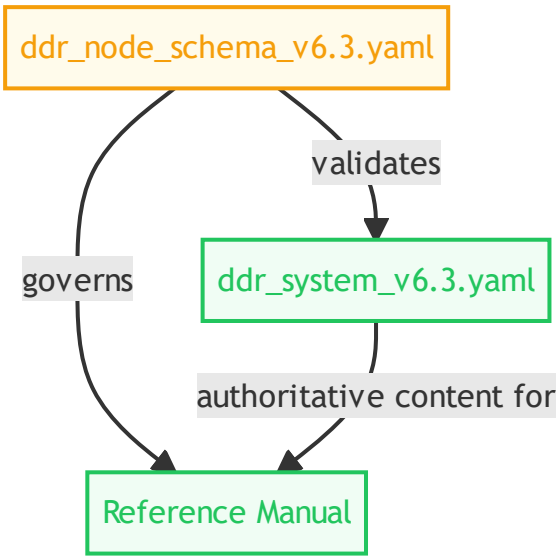
DDR System v6.3 Reference Manual

1. Governance and Scope

1.1 Authority model

SOURCE	ROLE	DESCRIPTION
`ddr_system_v6.3.yaml`	**Semantic Authority**	Semantic and structural authority for the DDR System definition
`ddr_node_schema_v6.3.yaml`	**Machine Contract**	Authority for allowed shapes, conditionals, enums, and validation branching

Figure 1.1. Authority precedence and manual role



Interpretation The manual depends on both YAML authorities at once: the system definition controls meaning, while the schema controls what valid DDR artifacts may look like.

1.2 Scope

PROPERTY	VALUE
DDR version	`6.3`
Document profile	`system_definition`
Project name	`DDR System v6.3 — Authoritative Specification`
Project mode	`full`
System status	`Finalized`
System date	`2026-03-28`
System scope	`Systems-, language-, and domain-agnostic`
Authority	`DDR Architecture Board`
Lineage	`Supersedes DDR v6.2`

Table of Contents

2. System Overview and Design Philosophy

2.1 System Metadata

SURFACE	VALUE
`project.name`	`DDR System v6.3 - Authoritative Specification`
`system_metadata.status`	`Finalized`
`system_metadata.single_source_of_truth`	This document is the exclusive normative specification.

3. Foundational Axioms and Core Structural Model

ID	NAME	STATEMENT	IMPLICATION
AX-1	**Traceability**	Every non-root node must cite at least one parent via a typed edge.	Complete audit trails from intent to implementation; no orphaned requirements.
AX-2	**Abstraction Ordering**	Technology and implementation specificity are deferred until logically necessary.	Tiers above CL (XPD, SIL, GPCL, FCL) must contain no technology, hardware, or implementation references.
AX-3	**Determinism**	Identical inputs produce unambiguous, mechanically verifiable outputs.	Automated validation and compliance checking are possible for all structural rules; semantic rules require explicit human disposition before node activation.
AX-4	**Universality**	The Core applies to all software systems regardless of domain, scale, or technology.	No domain-specific assumptions in any Core tier.
AX-5	**Extensibility**	Advanced analytical capabilities are delivered exclusively via optional Extensions.	Core structure remains stable and does not depend on Extension behavior. Extensions may interact with Core via explicitly defined, non-mutating interfaces.
AX-6	**Declarative Integrity**	The Core is strictly declarative; all inference, optimization, and automated recommendation are Extension-only behaviors.	Core structural invariants cannot be destabilized by analytical logic.
AX-7	**DAG Acyclicity**	No citation chain may produce a cycle; causality flows in one direction only.	Graph traversal is always terminable.

4. Tier Reference

Each tier is governed by explicit Inclusion (R) and Exclusion (E) rules.

Tier: XPD (Existential Purpose Document)

Core Question: What human or societal need does this system exist to address, and what ethical boundaries govern all downstream decisions?

RULE ID	STATEMENT
XPD-R1	Must articulate a fundamental human or societal need being addressed.
XPD-R2	Must be immutable across the project lifecycle; changes require a new XPD version.
XPD-R3	Must be comprehensible to non-technical stakeholders without a glossary.
XPD-R4	Must establish ethical boundary conditions all subsequent tiers must satisfy.
XPD-R5	Must define success criteria independent of implementation metrics.
XPD-R6	Must identify populations who could be harmed and the safeguards required.

XPD-E1	Must not contain solution concepts, technology references, or architectural ideas.
XPD-E2	Must not contain quantitative performance targets (→ GPCL).
XPD-E3	Must not contain regulatory or legal constraints (→ GPCL).

Tier: SIL (Strategic Intent Layer)

Core Question: Why does this system exist, and what business outcomes must it achieve?

RULE ID	STATEMENT
SIL-R1	Must define the core business problem or opportunity being addressed.
SIL-R2	Must specify strategic objectives with measurable outcomes.
SIL-R3	Must identify all stakeholder categories and their value propositions.
SIL-R4	Must establish explicit scope boundaries (in-scope and out-of-scope).
SIL-R5	Must define organizational success metrics.
SIL-R6	Must be stable under technology changes.
SIL-E1	Must not reference hardware, technology stacks, frameworks, or languages.
SIL-E2	Must not contain regulatory mandates or compliance requirements (→ GPCL).
SIL-E3	Must not prescribe architectural patterns or implementation strategies.
SIL-E4	Must not contain quantitative performance metrics (→ GPCL).

Tier: GPCL (Governance, Policy & Quality Layer)

Core Question: What non-negotiable external mandates, regulatory obligations, policy constraints, and measurable quality thresholds govern this system?

RULE ID	STATEMENT
GPCL-R1	Must enumerate all applicable regulatory frameworks with jurisdiction and scope.
GPCL-R2	Must specify enforceable, testable constraints — not aspirational targets.
GPCL-R3	Must identify contractual obligations imposed by third-party relationships.
GPCL-R4	Must define data sovereignty and residency requirements.
GPCL-R5	Must specify audit and record-retention mandates.
GPCL-R6	Must specify quantifiable performance targets: latency, throughput, concurrency ceilings.
GPCL-FCL-BR1	For every quantitative performance target specified under GPCL-R6, there must exist a corresponding FCL node whose semantic contribution is the behavioral context of the governed interaction — not a restatement of the numeric threshold. FCL nodes created solely to satisfy the citation chain without contributing independent behavioral context are prohibited. When no user-facing behavioral dimension exists for a GPCL performance target (e.g., infrastructure-level SLAs with no observable user interaction), the author must log a MISSING_MEDIATOR item to the reconciliation manifest. VERIFY must flag any direct GPCL→SAL dependency lacking an FCL mediator for human review.
GPCL-R7	Must specify reliability and availability targets (SLAs, RTO, RPO).
GPCL-R8	Must specify security requirements expressed technology-neutrally.
GPCL-R9	Must specify scalability and accessibility requirements.
GPCL-R10	Must cite parent SIL IDs for each constraint.

GPCL-E1	Must not specify technology frameworks, library choices, or hardware specifications.
GPCL-E2	Must not describe functional system behaviors (→ FCL).
GPCL-E3	Must not contain business objectives or success metrics (→ SIL).

Tier: FCL (Functional Capability Layer)

Core Question: What externally observable behaviors and user-facing capabilities must the system provide?

RULE ID	STATEMENT
FCL-R1	Must describe capabilities from the perspective of a user or external system.
FCL-R2	Must specify user workflows end-to-end without naming components, classes, or modules.
FCL-R3	Must define event-driven behaviors and conditional business logic rules.
FCL-R4	Must specify user-observable state transitions and error conditions.
FCL-R5	Must be decomposable into sub-capabilities (parent-child FCL nodes) for complex features.
FCL-R6	Must cite parent GPCL IDs for capabilities that satisfy a governance or quality requirement.
FCL-R7	For any capability that creates, reads, updates, or deletes persistent data, must enumerate all logical data entities involved by name and their CRUD relationship to the capability (for example: "creates Order, reads Customer, updates OrderItem"). Entity names must be technology-neutral logical identifiers. This rule requires entity names and CRUD verbs only and must not include attribute-level typing detail, storage-structure definitions, key declarations, or integrity-rule definitions; those belong in ICL and are prohibited at FCL level by FCL-E2.
FCL-E1	Must not name specific classes, modules, APIs, or algorithms.
FCL-E2	Must not specify network protocols, serialization formats, or data schemas.
FCL-E3	Must not specify hardware requirements or infrastructure topology.

Tier: CL (Constraint Layer)

Core Question: What are the declared technology selections, hardware envelopes, and infrastructure ceilings that bound the system's implementation?

RULE ID	STATEMENT
CL-R1	Must declare approved programming languages with version constraints.
CL-R2	Must declare mandatory frameworks and core libraries with minimum version bounds.
CL-R3	Must declare required external service contracts without their internal implementation details.
CL-R4	Must declare runtime environment constraints (OS, container runtime, execution environment).
CL-R5	Must explicitly declare prohibited technologies with rationale.
CL-R6	Must declare hardware envelopes when applicable (CPU class, RAM floor, storage, GPU).
CL-R7	Must declare infrastructure ceilings when applicable (compute budget, storage cap, bandwidth cap).
CL-R8	Must specify deployment topology declarations (on-premise, cloud-agnostic, hybrid, edge).
CL-R9	Must cite FCL IDs for each constraint.
CL-R9-imposed	Must cite the external authority source (regulatory framework, contract reference, procurement policy, or organizational mandate) that imposes the constraint. FCL citation is not required; an optional FCL cross-reference may be provided for contextual traceability.

CL-R10	Must explicitly document internal reconciliations of conflicting hardware and technology constraints.
CL-E1	Must not auto-derive, infer, or recommend configurations (→ Extensions).
CL-E2	Must not contain functional system behaviors (→ FCL).
CL-E3	Must not contain cost models or TCO calculations (→ Extensions).

Tier: SAL (System Architecture Layer)

Core Question: How is the system structurally decomposed, and what patterns govern component interaction?

RULE ID	STATEMENT
SAL-R1	Must define the overarching architectural pattern(s) with rationale.
SAL-R2	Must specify system decomposition into major subsystems with ownership boundaries.
SAL-R3	Must specify inter-subsystem communication patterns.
SAL-R4	Must specify concurrency model and data ownership rules.
SAL-R5	Must specify failure isolation and resilience boundaries.
SAL-R6	Must cite all active parent IDs (FCL + CL if active) for each major architectural decision.
SAL-E1	Must not contain exact data schemas or payload definitions (→ ICL).
SAL-E2	Must not contain class-level component blueprints (→ CDL).
SAL-E3	Must not contain executable code, algorithm implementations, or procedural logic (→ CDL/ISL).

Tier: ICL (Interface & Contracts Layer)

Core Question: What are the formal, machine-verifiable contracts governing data exchange between system boundaries?

RULE ID	STATEMENT
ICL-R1	Must define all inter-component and external API contracts with complete input/output schemas.
ICL-R2	All schemas must be machine-parseable (JSON Schema, Protobuf, OpenAPI, or equivalent).
ICL-R3	Must specify serialization formats, encoding standards, and wire protocols per contract.
ICL-R4	Must specify mandatory fields, optional fields, type constraints, and validation rules.
ICL-R5	Must specify error response contracts (error codes, payload structure, retry behavior).
ICL-R6	Must specify versioning strategy per contract.
ICL-R7	Must cite SAL IDs for each contract.
ICL-E1	Must not contain internal component state management or business logic.
ICL-E2	Must not specify architectural routing patterns (→ SAL).
ICL-E3	Must not contain class or module blueprints (→ CDL).

Tier: CDL (Component Design Layer)

Core Question: What are the structural blueprints of individual components — their public interfaces, internal state, and responsibilities?

RULE ID	STATEMENT

CDL-R1	Must define component names, logical responsibilities, and ownership boundaries.
CDL-R2	Must specify all public method/function signatures (name, parameter types, return type, exceptions).
CDL-R3	Must specify internal state structures as a logical model — not implementation.
CDL-R4	Must specify component dependencies (consumed components and ICL contracts).
CDL-R5	Must map each component to the ICL contracts it implements.
CDL-R6	Must specify initialization, lifecycle, and teardown contracts for stateful components.
CDL-R7	When CL declares multiple target languages, must produce language-specific blueprints for each target.
CDL-E1	Must not contain executable code bodies or algorithm implementations.
CDL-E2	Must not contain system-wide architectural patterns (→ SAL).
CDL-E3	Must not contain data serialization schemas (→ ICL).

Tier: ISL (Implementation Scaffold Layer)

Core Question: What is the minimal, structurally valid, traceable scaffolding required to initiate implementation?

RULE ID	STATEMENT
ISL-R1	Must produce syntactically valid structural scaffolding in the target language.
ISL-R2	Must embed docstrings or code comments with explicit parent DDR node IDs.
ISL-R3	Must include implementation hints as structured comments.
ISL-R4	Must define all function/method bodies exclusively as stubs.
ISL-R5	Must be language-specific — one ISL node per target language/runtime when multiple are declared in CL.
ISL-R6	Must cite CDL parent IDs for every stub.
ISL-E1	Must not contain business logic or complete algorithmic logic.
ISL-E2	Must not contain infrastructure configuration (→ Extensions).